

Spring 2026 COP 3502H Final Exam Part A 4/30/2026 Solutions

1) (10 pts) Complete the function below which uses the binary search method to determine the n^{th} root of x . Both x and n are passed in as parameters. x is guaranteed to be a positive real number less than one billion and n is guaranteed to be a positive integer greater than 1. Note: be careful setting low and high. Also, it's intended for you to use a call to a function in the math library in your solution.

```
#include <math.h>

double nthRoot(double x, int n) {

    double low = 1, high = n;           // 1 pt for these bounds.

    if (x < 1) {                         // 2 pts for this case.
        high = 1;
        low = x;
    }

    // 1 pt will accept epsilon while loop also...
    for (int i=0; i<100; i++) {

        double mid = (low+high)/2;       // 1 pt
        double res = pow(mid, n);       // 2 pts

        if (res < x)                    // 1 pt each branch
            low = mid;
        else
            high = mid;
    }

    return low;                         // 1 pt
}
```

2) (15 pts) Complete the code on the next page so that it prints out the first jumping knight tour that visits n unique locations. In your solution, you must fill in the given DR, DC arrays to follow how a knight in chess jumps. Most of what this question is testing is your ability to read the portions of the code given to understand how the rest of the solution must work. The variables are named normally and are NOT obfuscating their role, so use that to your advantage. The grid is input as several strings, each of length COLS. Each character is either 'S', '_' or 'X'. There is exactly one 'S' character. This indicates the starting location. '_' are valid locations for the knight to jump to. 'X' are invalid locations. For the input grid

S__X	the program prints out
X__X_	(0, 0) -> (1, 2) -> (2, 0) -> (0, 1) -> (2, 2) -> (0, 3) -> (1, 1)

_X__	For this code, the # of rows is 4 and # cols is 5 (and is hard-coded)

All support code is on the back of this sheet.

```

#include <stdio.h>
#include <stdlib.h>

#define ROWS 4
#define COLS 5
#define LEN 6
#define NUMDIR 8

const int DR[] = {-2,-2,-1,-1,1,1,2,2};
const int DC[] = {-1,1,-2,2,-2,2,-1,1};

int go(char board[][COLS+1], int used[][COLS], int* path, int curLoc, int k, int n);
int inbounds(int r, int c);
void init(int used[][COLS], char board[][COLS+1]);
int findS(char board[][COLS+1]);

int main() {

    char board[ROWS][COLS+1];
    for (int i=0; i<ROWS; i++)
        scanf("%s", board[i]);

    int used[ROWS][COLS];
    init(used, board);
    int loc = findS(board);

    int path[ROWS*COLS];
    path[0] = loc;
    go(board, used, path, loc, 1, LEN+1);

    for (int i=0; i<LEN; i++)
        printf("(%d,%d)->", path[i]/COLS, path[i]%COLS);
    printf("(%d,%d)\n", path[LEN]/COLS, path[LEN]%COLS);
    return 0;
}

void init(int used[][COLS], char board[][COLS+1]) {
    for (int i=0; i<ROWS; i++)
        for (int j=0; j<COLS; j++)
            used[i][j] = (board[i][j] == 'X' || board[i][j] == 'S');
}

int findS(char board[][COLS+1]) {
    for (int i=0; i<ROWS; i++)
        for (int j=0; j<COLS; j++)
            if (board[i][j] == 'S')
                return i*COLS + j;
    return -1;
}

int inbounds(int r, int c) {
    return r>=0 && r<ROWS && c>=0 && c<COLS;
}

```

```

int go(char board[][COLS+1], int used[][COLS], int* path, int curLoc, int k, int n)
{
    if (k == n) return 1;

    int curR = curLoc/COLS; // 1 pt
    int curC = curLoc%COLS; // 1 pt
    for (int i=0; i<NUMDIR; i++) {
        int nextR = curR + DR[i] ; // 1 pt

        int nextC = curC + DC[i]; // 1 pt

        if ( inbounds(nextR, nextC) ) continue; // 2 pts

        if ( used[nextR][nextC]) continue; // 1 pts

        int nextLoc = nextR*COLS + nextC; // 2 pts
        used[nextR][nextC] = 1; // 1 pt

        path[k] = nextLoc; // 1 pt

        int tmp = go(board, used, path, nextLoc, k+1, n);

        if (tmp) return 1; // 2 pts

        used[nextR][nextC] = 0; // 1 pt
    }
    return 0; // 1 pt
}

```